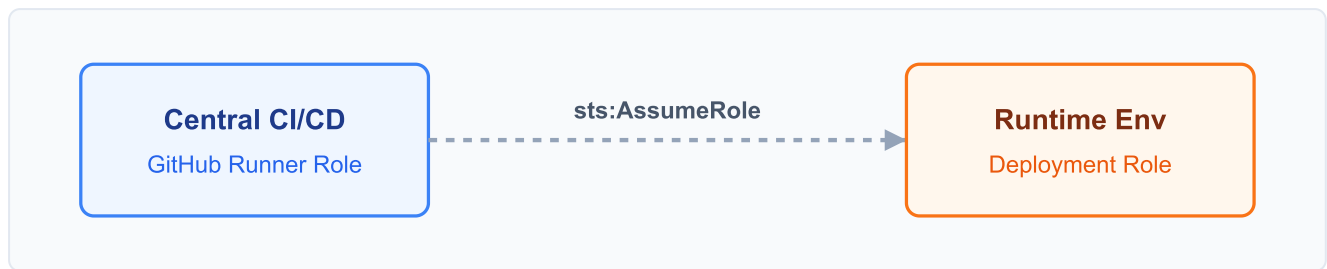


AWS Cross-Account CI/CD Architecture

To securely deploy resources from a single central CI/CD account into multiple runtime environments (Dev, Acceptance, Prod), AWS uses the **IAM AssumeRole** pattern. This allows your self-hosted GitHub Actions runner to temporarily request the necessary permissions in the target accounts without hardcoding or sharing long-term access keys.

Here is the architectural flow of how the central runner accesses your runtime environments:



Here is the exact step-by-step configuration to implement this architecture securely.

Step 1: Create IAM Roles in the Target Accounts (Dev, Acc, Prod)

In each of your runtime accounts, you must create a specific "Deployment Role."

1. The Trust Policy (Who can access this?)

This policy goes on the target role and explicitly "trusts" your central CI/CD account. You can scope this down to the specific IAM Role your self-hosted runner uses.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<CICD_ACCOUNT_ID>:role/<GitHub_Runner_Role_Name>"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

2. The Permissions Policy (What can it do?)

Attach standard IAM permission policies to this Deployment Role in the target account. If the pipeline needs to upload to S3 and update a Lambda function, attach policies granting `s3:PutObject`, `lambda:UpdateFunctionCode`, etc.

Step 2: Configure the Central CI/CD Account

Your GitHub self-hosted runner is running on compute (like EC2 or ECS) inside the Central Account. That compute instance has an IAM Role attached to it.

You must give the runner's IAM Role permission to reach out and "assume" the target roles you created in Step 1. Attach this policy to the runner's role in the CI/CD account:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": [
        "arn:aws:iam::<DEV_ACCOUNT_ID>:role/<Dev_Deployment_Role>",
        "arn:aws:iam::<ACC_ACCOUNT_ID>:role/<Acc_Deployment_Role>",
        "arn:aws:iam::<PROD_ACCOUNT_ID>:role/<Prod_Deployment_Role>"
      ]
    }
  ]
}
```

Step 3: Configure Your GitHub Actions Pipeline

When a developer pushes code, the GitHub Action triggers on your self-hosted runner. Because the runner already has the base permissions from its EC2/ECS instance profile, you just need to tell it which target account role to assume for that specific deployment.

Use the official `aws-actions/configure-aws-credentials` step in your workflow file. You can dynamically pass the target Account ID based on the environment being built:

YAML

```
jobs:
  deploy:
    runs-on: self-hosted
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Configure AWS Credentials for Target Account
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-region: us-east-1
          role-to-assume: arn:aws:iam::${{ env.TARGET_ACCOUNT_ID }}:role/DeploymentRole
          role-session-name: GitHubActionsDeploySession

      - name: Deploy to S3
        run: aws s3 cp /build/ s3://my-target-bucket/ --recursive
```